

z/OS Job Control Language (JCL)

GENERATION DATA GROUPS





GENERATION DATA GROUPS

Imagine a mid-20th-century data processing shop that ran a number of high-volume batch jobs every night to prepare for the next day's business. Most of the files were sequential; in the early days of mainframes, they were on tape. Failures and restarts in these batch jobs were common.

Under those conditions, it would be handy to be able to version the files to facilitate restarting processing somewhere other than the very beginning of each job.



GENERATION DATA GROUPS

Some systems of the day, like Digital Equipment Corporation's VAX and its VMS operating system, automatically versioned every file by default. But the IBM System/360 did not.

IBM's answer to this need was the *Generation Data Group* (GDG), a special type of system catalog object that keeps track of multiple versions of the "same" versioned data set, called a *Generation Data Set* (GDS). JCL can refer to any existing version, for purposes of job restart and data recovery.



DEC VAX 11/780 (1970s)

GENERATION DATA GROUPS

SOME .DATA .SET (+1)	SOME .DATA .SET .G0100V00	Next version to be created
SOME .DATA .SET (0)	SOME .DATA .SET .G0099V00	Current version
SOME .DATA .SET (-1)	SOME .DATA .SET .G0098V00	One version back
SOME .DATA .SET (-2)	SOME .DATA .SET .G0097V00	Two versions back
. . .		
SOME .DATA .SET (-98)	SOME .DATA .SET .G0002V00	
SOME .DATA .SET (-99)	SOME .DATA .SET .G0001V00	First version

JCL refers to GDSs by their relative version number, where version zero is the latest or current version of the data set.

Earlier versions of the data set have negative relative numbers.

To create a new GDS, you code a relative version number of +1.

The system creates a separate catalogued data set for each version.

If we modify GDSs using their actual data set names, we circumvent the GDG logic and the system may lose track of the versions.

GENERATION DATA GROUPS

BSAM	Yes
QSAM	Yes
BDAM	Yes
PDS	Yes
PDSE	Yes
VSAM	No

A GDG can manage most of the traditional types of z/OS data set types, but not VSAM types.

THE IDCAMS UTILITY

Generation Data Groups are defined using another IBM utility – IDCAMS.

IDC is the three-letter product prefix, and AMS stands for *Access Method Services*.

IDCAMS is used to create and manage GDGs as well as all types of VSAM data sets.

Some people pronounce IDCAMS as "idd-cams," and other as "eye-dee-cams".



CREATING AND USING A GDG

There's a three-step process to create a GDG and the first GDS in the GDG:

1. Use IDCAMS to define and catalog the GDG.
2. Use IEFBR14 and a DD statement to create a model DCB for GDSs that will belong to the GDG.
3. Use DD statements as you would to create any data set, but with (+1) appended to the data set name to cause the system to create a new GDS in the GDG.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBMUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
    DEFINE GDG ( -  
        NAME(IBMUSER.SAMPLE.GDG) -  
        LIMIT(50) -  
        NOEMPTY -  
        SCRATCH -  
    )  
/*
```

Let's look at the sample job to create a Generation Data Group and a model DSCB.

The EXEC statement for the first job step specifies program IDCAMS, the Access Method Services utility.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN     DD *  
DELETE IBUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

IDCAMS operates by processing commands that are supplied to it in the SYSIN data set. SYSIN can be a catalogued data set or an inline data set. In our sample JCL, it's an inline data set.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBMUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

It's convenient when this kind of job is idempotent – that is, running it multiple times will produce the same result as running it once.

We also don't want the job to fail if the GDG already exists – we want to delete it and create it fresh in that case.

So the IDCAMS commands start with a DELETE. The PURGE option gets rid of any GDSs that belong to the GDG, and then deletes the GDG catalog entry.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBMUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
    DEFINE GDG ( -  
        NAME(IBMUSER.SAMPLE.GDG) -  
        LIMIT(50) -  
        NOEMPTY -  
        SCRATCH -  
    )  
/*
```

When IDCAMS attempts to delete a data set or catalog entry that doesn't exist, it returns condition code 8.

In this case, we want to proceed with the DEFINE command if the condition code is 8. Values higher than 8 denote genuine failures, and we should terminate execution.

The DEFINE command is part of the same statement as the IF command, so we code a dash to indicate "continuation" to the command parser.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
/* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBMUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

The DEFINE command tells IDCAMS to create a catalog entry for a GDG. We could also write GENERATIONDATAGROUP instead of GDG, but...you know.

The catalog entry for a GDG doesn't point to an actual data set. It's a reference for managing the individual Generation Data Sets that belong to this group.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID
//*****
/* Catalog a Generation Data Group, deleting the old one if it exists.
//*****
//CATGDG   EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN    DD *
DELETE IBUSER.SAMPLE.GDG PURGE
IF LASTCC < 9 THEN -
  DEFINE GDG ( -
    NAME(IBMUSER.SAMPLE.GDG) -
    LIMIT(50) -
    NOEMPTY -
    SCRATCH -
  )
/*
```

The NAME parameter specifies the name of the GDG. Unfortunately, it isn't possible to use symbolics within the SYSIN data set, so we have to hard-code the high-level qualifier in all the IDCAMS control statements that need it.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

The LIMIT parameter specifies the number of generations of Generation Data Sets to retain. Here we're telling IDCAMS we want to keep the latest 50 generations.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

NOEMPTY means that when the 51st generation of data sets is created, we don't want the system to clear out the GDG. Instead, we want to roll off the oldest generation and keep the latest 50.

CREATING AND USING A GDG

```
//IBUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Catalog a Generation Data Group, deleting the old one if it exists.  
//*****  
//CATGDG   EXEC PGM=IDCAMS  
//SYSPRINT DD SYSOUT=*  
//SYSIN    DD *  
DELETE IBUSER.SAMPLE.GDG PURGE  
IF LASTCC < 9 THEN -  
  DEFINE GDG ( -  
    NAME(IBMUSER.SAMPLE.GDG) -  
    LIMIT(50) -  
    NOEMPTY -  
    SCRATCH -  
  )  
/*
```

SCRATCH means that when the oldest generation is rolled off, we want to delete it from the system. Otherwise, the data set would still exist, but would no longer be processed as part of the GDG.

CREATING AND USING A GDG

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSERG JOB00622 DSID      2 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █      SCROLL ==> CSR
***** TOP OF DATA *****
      JES2 JOB LOG -- SYSTEM VS01 -- N

11.52.19 JOB00622 ---- TUESDAY, 27 JAN 2026 ----
11.52.19 JOB00622 IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
11.52.19 JOB00622 ICH70001I IBMUSER LAST ACCESS AT 11:50:21 ON TUESDAY, JANUAR
11.52.19 JOB00622 $HASP373 IBMUSERG STARTED - INIT 1 - CLASS A - SYS
11.52.19 JOB00622 IEF403I IBMUSERG - STARTED - TIME=11.52.19
11.52.19 JOB00622 IEF404I IBMUSERG - ENDED - TIME=11.52.19
11.52.19 JOB00622 $HASP395 IBMUSERG ENDED - RC=0008
----- JES2 JOB STATISTICS -----
      27 JAN 2026 JOB EXECUTION DATE
          17 CARDS READ
          65 SYSOUT PRINT RECORDS
           0 SYSOUT PUNCH RECORDS
           7 SYSOUT SPOOL KBYTES
          0.00 MINUTES EXECUTION TIME
***** B
```

Let's examine the system output from the job and see what it tells us.

The JESMSGGLG shows the job completed with a maximum condition code of 8, which means a non-fatal error occurred. Did you expect this? Why or why not?

CREATING AND USING A GDG

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSERG JOB00622 DSID      3 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █      SCROLL ==> CSR
***** TOP OF DATA *****
 1 //IBMUSERG JOB , 'CATLG GDG', CLASS=A, MSGCLASS=X,
  //          MSGLEVEL=(1,1), NOTIFY=&SYSUID
  //*****
  // * Catalog a Generation Data Group, deleting the old one if it exists
  //*****
  IEFC653I SUBSTITUTION JCL - , 'CATLG GDG', CLASS=A, MSGCLASS=X, MSGLEVEL=(
 2 //CATGDG      EXEC PGM=IDCAMS
 3 //SYSPRINT    DD SYSOUT=*
 4 //SYSIN       DD *
***** BOTTOM OF DATA *****
```

The JESJCL data set shows the JCL for the job, but not the IDCAMS commands, because they are data within the SYSIN data set, and not JCL statements.

CREATING AND USING A GDG

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSERG JOB00622 DSID      4 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █
***** TOP OF DATA *****
ICH70001I IBMUSER  LAST ACCESS AT 11:50:21 ON
IEFA111I IBMUSERG IS USING THE FOLLOWING JOB R
        SWA=BELOW,TIOT SIZE=64K,DSENQSHR=DISA
IEF236I ALLOC. FOR IBMUSERG CATGDG
IEF237I JES2 ALLOCATED TO SYSPRINT
IEF237I JES2 ALLOCATED TO SYSIN
IEF142I IBMUSERG CATGDG - STEP WAS EXECUTED - COND CODE 0008
IEF285I  IBMUSER.IBMUSERG.JOB00622.D0000102.?      SYSOUT
IEF285I  IBMUSER.IBMUSERG.JOB00622.D0000101.?      SYSIN
IEF373I STEP/CATGDG  /START 2026027.1152
IEF032I STEP/CATGDG  /STOP 2026027.1152
        CPU:      0 HR  00 MIN  00.03 SEC      SRB:      0 HR  00 MIN  00.00 SEC
        VIRT:    216K  SYS:    308K  EXT:      32K  SYS:    10156K
        ATB- REAL:                1252K  SLOTS:                0K
        VIRT- ALLOC:      14M  SHRD:      0M
IEF375I  JOB/IBMUSERG/START 2026027.1152
IEF033I  JOB/IBMUSERG/STOP 2026027.1152
        CPU:      0 HR  00 MIN  00.03 SEC      SRB:      0 HR  00 MIN  00.00 SEC
***** BOTTOM OF DATA *****
```

The JESYSMSG data set shows the condition code was 8, but otherwise isn't very clear about what happened. Was the GDG catalogued or not?

COND CODE 0008

CREATING AND USING A GDG

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSERG JOB00622 DSID 102 LINE 0 COLUMNS 02- 81
COMMAND INPUT ==> █ SCROLL ==> CSR
***** TOP OF DATA *****
IDCAMS SYSTEM SERVICES TIME: 11:52:19

DELETE IBMUSER.SAMPLE.GDG PURGE
IDC3012I ENTRY IBMUSER.SAMPLE.GDG NOT FOUND
IDC3009I ** VSAM CATALOG RETURN CODE IS 8 - REASON CODE IS IGG0CLEG-42
IDC0551I ** ENTRY IBMUSER.SAMPLE.GDG NOT DELETED
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 8

IF LASTCC < 9 THEN -
  DEFINE GDG ( -
    NAME(IBMUSER.SAMPLE.GDG) -
    LIMIT(50) -
    NOEMPTY -
    SCRATCH -
  )
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM C
***** BOTTOM OF DATA *****
```

The SYSPRINT data set is produced by IDCAMS, not by JES. This is where to find information about the results of the IDCAMS commands.

The GDG was not found, so the DELETE command failed. That's what we expected – and wanted.

CREATING AND USING A GDG

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSERG JOB00622 DSID 102 LINE 0 COLUMNS 02- 81
COMMAND INPUT ==> SCROLL ==> CSR
***** TOP OF DATA *****
IDCAMS SYSTEM SERVICES TIME: 11:52:19

DELETE IBMUSER.SAMPLE.GDG PURGE
IDC3012I ENTRY IBMUSER.SAMPLE.GDG NOT FOUND
IDC3009I ** VSAM CATALOG RETURN CODE IS 8 - REASON CODE IS IGG0CLEG 42
IDC0551I ** ENTRY IBMUSER.SAMPLE.GDG NOT DELETED
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 8

IF LASTCC < 9 THEN -
  DEFINE GDG ( -
    NAME(IBMUSER.SAMPLE.GDG) -
    LIMIT(50) -
    NOEMPTY -
    SCRATCH -
  )
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 8
***** BOTTOM OF DATA *****
```

The IF condition passed, so the DEFINE command was processed.

The condition code for the DEFINE command was zero – it worked.

CREATING AND USING A GDG

```
Menu  RefList  Utilities  Help

Data Set Utility

Option ==> v

  A Allocate new data set      C Catalog data set
  R Rename entire data set    U Uncatalog data set
  D Delete entire data set    S Short data set information
blank Data set information    V VSAM Utilities

ISPF Library:
Project . . . ibmuser
Group   . . . sample
Type    . . . gdg

Enter "/" to select option
/ Confirm Data Set Delete

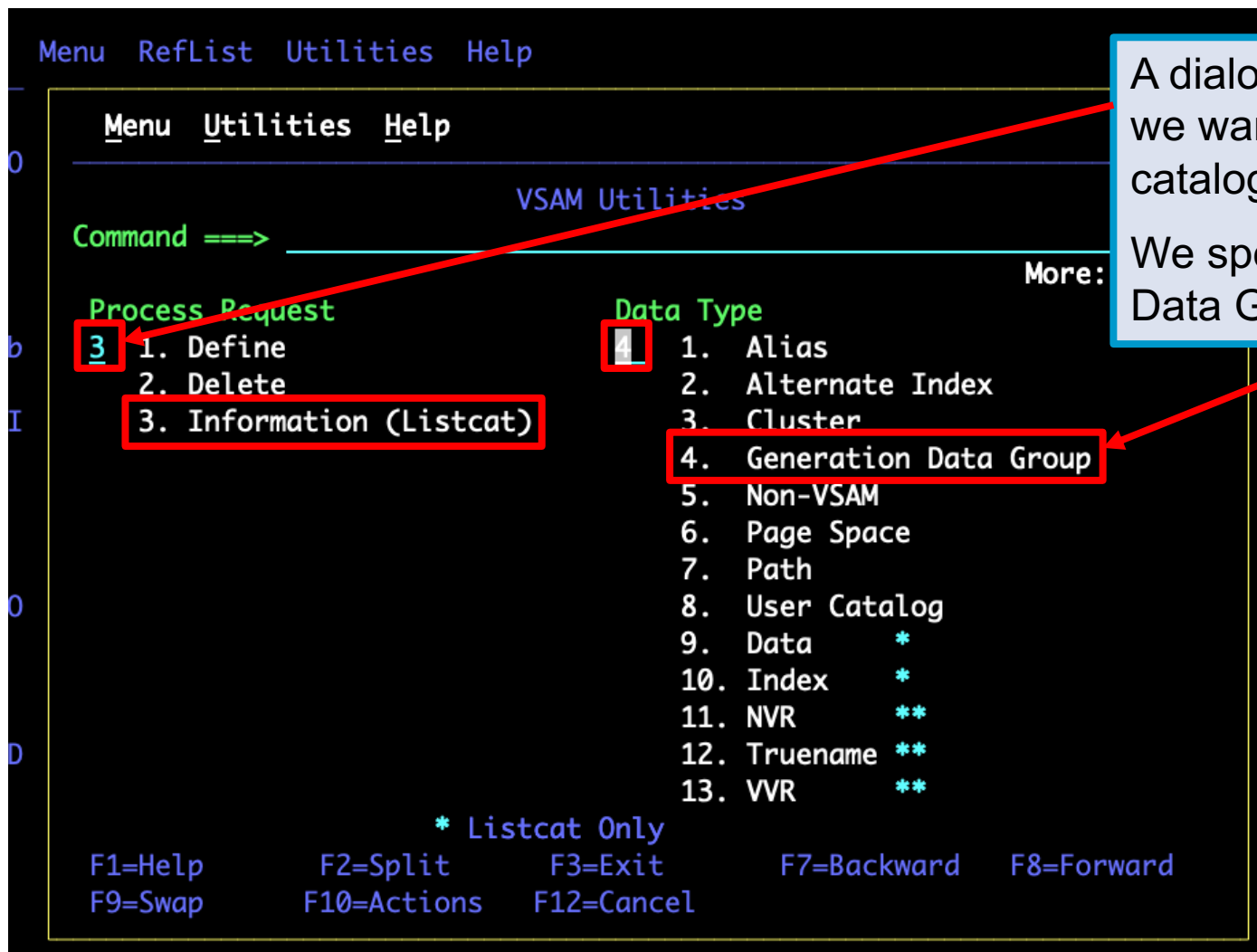
Other Partitioned, Sequential or VSAM Data Set:
Name . . . . .
Volume Serial . . . (If not cataloged, required for d
Data Set Password . . (If password protected)
```

To be really sure it worked, let's check the catalog entry.

We start from the ISPF Data Set Utility panel, usually reached by option 3.2 from the Primary Option Menu.

We enter option "v" for VSAM and fill in the catalog entry name. For this example, we used the ISPF Library fields instead of the "other" field, since the DSN has 3 segments.

CREATING AND USING A GDG



A dialog pops up prompting us for the action we want (Process Request) and the type of catalog entry we're interested in (Data Type).

We specify "Information" and "Generation Data Group" and press Enter.

CREATING AND USING A GDG

```
VSAM Utilities ----- LINE 00000000 COL 001 080
COMMAND ==> | SCROLL ==> PAGE
***** Top of Data *****
IDCAMS  SYSTEM SERVICES                                TIME: 14:13:59

/* IDCAMS COMMAND */
LISTCAT ENTRIES(IBMUSER.SAMPLE.GDG) -
      GENERATIONDATAGROUP
GDG BASE ----- IBMUSER.SAMPLE.GDG
IN-CAT --- CATALOG.VS01.TSO
IDCAMS  SYSTEM SERVICES                                TIME: 14:13:59
      THE NUMBER OF ENTRIES PROCESSED WAS:
      AIX -----0
      ALIAS -----0
      CLUSTER -----0
      DATA -----0
      GDG -----1
      INDEX -----0
      NONVSAM -----0
      PAGESPACE -----0
      PATH -----0
      SPACE -----0
      USERCATALOG -----0
      TAPELIBRARY -----0
      TAPEVOLUME -----0
      TOTAL -----1
      THE NUMBER OF PROTECTED ENTRIES SUPPRESSED WAS 0
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
IDC0002I IDCAMS PROCESSING COMPLETE. MAXIMUM CONDITION CODE WAS 0
F1=Help  F2=Split  F3=Exit  F5=Rfind  F7=Up  F8=Down  F9=Swap
F10=Left F11=Right F12=Cancel
```

You can see this runs IDCAMS in the background. We could have run the same query in a batch job that looked pretty similar to our JCL to create the GDG. We could also have entered LISTCAT at the TSO ready prompt. There's more than one way to do it.

One catalog entry was found that matched our criteria – name = IBMUSER.SAMPLE.GDG and type = GDG

LABS 21 AND 22: CREATING A GDG

Follow the instructions for Labs 21 and 22 in labs/labs.md in the course repo.

WHAT IS A MODEL DSCB?

In almost all cases, the individual Generation Data Sets in any given Generation Data Group have the same Data Control Block attributes. This isn't enforced by the system, but it would make little sense for each GDS to have a unique record format or record length, and still less a unique data set type.

When jobs create new GDSs during processing, if the DD statements had to duplicate the same attributes there would be a good chance of errors. To ensure all new GDSs have the same DCB attributes, we use a Model DSCB (Model Data Set Control Block).

A Model DSCB is just a catalog entry where the DCB attributes are stored. It's the same as the catalog entry for a "normal" data set. There's nothing special about the Model DSCB, except that it doesn't occupy any space on DASD. It isn't used as a "file," but only as a reference in DD statements that create new data sets with the same attributes.

Let's see how that looks.

CREATING A MODEL DSCB

```
//IBUSERG JOB , 'MODEL DSCB', CLASS=A, MSGCLASS=X,  
//          MSGLEVEL=(1,1), NOTIFY=&SYSUID  
//*****  
//* Create a model DSCB for a QSAM data set.  
//*****  
//MODDSCB EXEC PGM=IEFBR14  
//DD1      DD DSN=&SYSUID..MODEL.DSCB120,  
//          DISP=(MOD,CATLG,DELETE),  
//          DCB=(DSORG=PS, RECFM=FB, LRECL=120, BLKSIZE=24000),  
//          SPACE=(TRK,(0,0)),  
//          UNIT=SYSDA
```

This job runs the do-nothing utility IEFBR14, specifying a DD statement to create a catalog entry for a QSAM data set with fixed-blocked records, logical record length 120 and block size 24000. Notice that no space is allocated, because we aren't going to use this as a "normal" data set. The catalog entry can be referenced in DD statements for new data sets that need the same attributes.

LABS 23 AND 24: CREATING A MODEL DSCB

Follow the instructions for Labs 23 and 24 in labs/labs.md in the course repo.

CREATING GENERATION DATA SETS

Now when application jobs need to add a versioned data set to the GDG, they can specify the model DSCB instead of hard-coding the data set attributes.



CREATING GENERATION DATA SETS

```
//IBUSER2 JOB , 'HAMLET', CLASS=A,  
//      MSGCLASS=X,  
//      MSGLEVEL=(1,1),  
//      NOTIFY=&SYSUID  
//SOLIL1 EXEC PGM=IEBGENER  
//SYSIN  DD DUMMY  
//SYSOUT DD SYSOUT=*  
//SYSPRINT DD SYSOUT=*  
//SYSUT1 DD *  
To be, or not to be, that is the question:  
Whether 'tis nobler in the mind to suffer  
The slings and arrows of outrageous fortune  
Or to take arms against a sea of troubles  
And by opposing, end them.  
/*  
//SYSUT2 DD DSN=&SYSUID..TEST.GDG(+1),  
//      DCB=&SYSUID..MODEL.DSCB80,  
//      DISP=(NEW,CATLG,DELETE),  
//      SPACE=(TRK,(1,1),RLSE),  
//      UNIT=SYSDA
```

This JCL uses IEBGENER to copy text from an inline SYSIN data set to the next generation of a GDG.

The SYSUT2 DD statement specifies (+1) at the end of the DSN.

It also references the model DSCB rather than hard-coding DCB attributes.

Disposition status is NEW because the GDS is new; it doesn't refer to the base GDG.

CREATING GENERATION DATA SETS

```
//IBUSER3 JOB , 'HAMLET', CLASS=A,
//      MSGCLASS=X,
//      MSGLEVEL=(1,1),
//      NOTIFY=&SYSUID
//SOLIL1 EXEC PGM=IEBGENER
//SYSIN  DD DUMMY
//SYSOUT DD SYSOUT=*
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD *
To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune
Or to take arms against a sea of troubles
And by opposing, end them.
/*
//SYSUT2 DD DSN=&SYSUID..TEST.GDG(+1),
//      DCB=&SYSUID..MODEL.DSCB80,
//      DISP=(NEW,CATLG,DELETE),
//      SPACE=(TRK,(1,1),RLSE),
//      UNIT=SYSDA
//SOLIL2 EXEC PGM=IEBGENER
//SYSIN  DD DUMMY
//SYSOUT DD SYSOUT=*
//SYSPRINT DD SYSOUT=*
//SYSUT1 DD *
To die: To sleep; no more.
And by a sleep we say to end the heart-ache
and the thousand natural shocks that flesh is hier to,
'tis a consummation devoutly to be wished.
/*
//SYSUT2 DD DSN=&SYSUID..TEST.GDG(+2),
//      DCB=&SYSUID..MODEL.DSCB80,
//      DISP=(NEW,CATLG,DELETE),
//      SPACE=(TRK,(1,1),RLSE),
//      UNIT=SYSDA
```

This example contains two job steps, each of which writes the next generation of the same GDG.

The key thing to notice here is that the second step must specify the (+2) generation. This is because the updated generation numbers occur at the end of the job, and not at the end of each step. So the "current" generation, generation (0), refers to the same GDS in both steps.

CREATING GENERATION DATA SETS

```
VIEW          IBMUSER.TEST.GDG.G0002V00          Columns 00001 00072
Command ==> |                                     Scroll ==> CSR
***** ***** Top of Data *****
000001 To be, or not to be, that is the question:
000002 Whether 'tis nobler in the mind to suffer
000003 The slings and arrows of outrageous fortune
000004 Or to take arms against a sea of troubles
000005 And by opposing, end them.
***** ***** Bottom of Data *****
```

Generation (-2)
Written by GENER2

```
VIEW          IBMUSER.TEST.GDG.G0003V00          Columns 00001 00072
Command ==> |                                     Scroll ==> CSR
***** ***** Top of Data *****
000001 To be, or not to be, that is the question:
000002 Whether 'tis nobler in the mind to suffer
000003 The slings and arrows of outrageous fortune
000004 Or to take arms against a sea of troubles
000005 And by opposing, end them.
***** ***** Bottom of Data *****
```

Generation (-1)
Written by GENER3,
step SOLIL1

```
VIEW          IBMUSER.TEST.GDG.G0004V00          Columns 00001 00072
Command ==> |                                     Scroll ==> CSR
***** ***** Top of Data *****
000001 To die: To sleep; no more.
000002 And by a sleep we say to end the heart-ache
000003 and the thousand natural shocks that flesh is hier to,
000004 'tis a consummation devoutly to be wished.
***** ***** Bottom of Data *****
```

Generation (0)
Written by GENER3,
step SOLIL2

READING GENERATION DATA GROUPS

```
//IBMUSER4 JOB , 'READ GDG', CLASS=A,  
//          MSGCLASS=X,  
//          MSGLEVEL=(1,1),  
//          NOTIFY=&SYSUID  
//READGDG  EXEC PGM=IEBGENER  
//SYSIN    DD DUMMY  
//SYSOUT   DD SYSOUT=*  
//SYSPRINT DD SYSOUT=*  
//SYSUT1   DD DSN=&SYSUID..TEST.GDG,  
//          DISP=SHR  
//SYSUT2   DD SYSOUT=*
```

When reading a GDG, if you specify the DSN of the GDG without a generation number, the system returns all generations concatenated as a single data set, in reverse order – that is, from newest to oldest.

Use DISP=SHR because you don't need an exclusive lock for read-only access.

This example directs SYSUT2 to SYSOUT.

```
Display Filter View Print Options Search Help  
-----  
SDSF OUTPUT DISPLAY IBMUSER4 JOB00833 DSID 103 LINE  
COMMAND INPUT ==>   
***** TOP OF DATA *****  
To die: To sleep; no more.  
And by a sleep we say to end the heart-ache  
and the thousand natural shocks that flesh is hier to,  
'tis a consummation devoutly to be wished.  
To be, or not to be, that is the question:  
Whether 'tis nobler in the mind to suffer  
The slings and arrows of outrageous fortune  
Or to take arms against a sea of troubles  
And by opposing, end them.  
To be, or not to be, that is the question:  
Whether 'tis nobler in the mind to suffer  
The slings and arrows of outrageous fortune  
Or to take arms against a sea of troubles  
And by opposing, end them.  
To be, or not to be, that is the question:  
Whether 'tis nobler in the mind to suffer  
The slings and arrows of outrageous fortune  
Or to take arms against a sea of troubles  
And by opposing, end them.  
***** BOTTOM OF DATA *****
```

READING GENERATION DATA GROUPS

```
//IBMUSER4 JOB , 'READ GDG', CLASS=A,
//          MSGCLASS=X,
//          MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
//READGDG  EXEC PGM=IEBGENER
//SYSIN    DD DUMMY
//SYSOUT   DD SYSOUT=*
//SYSPRINT DD SYSOUT=*
//SYSUT1   DD DSN=&SYSUID..TEST.GDG(-2),
//          DISP=SHR
//          DD DSN=&SYSUID..TEST.GDG(0),
//          DISP=SHR
//SYSUT2   DD SYSOUT=*
```

DD concatenation

from (0) from (-2)

```
Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER4 JOB00834 DSID 103 LINE
COMMAND INPUT ==>
***** TOP OF DATA *****
To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune
Or to take arms against a sea of troubles
And by opposing, end them.
To die: To sleep; no more.
And by a sleep we say to end the heart-ache
and the thousand natural shocks that flesh is hier to,
'tis a consummation devoutly to be wished.
```

To get one or more specific generations, code the relative generation number in parentheses at the end of each DSN.

You could code the actual DSNs, like &SYSUID..TEST.GDG.G0037V00, but doing so can confuse the system and mess up the management of the generations. (Yes, the GDG feature is a little flaky.)

This example shows how to concatenate multiple data sets under a single DD statement.

This will be relevant to your next lab exercise, because you can't do this purely with Ansible. You will have to call on your JCL and Ansible skills together.

LAB 25: FETCH CONCATENATED GDS'S

Follow the instructions for Lab 25 in labs/labs.md in the course repo.

MORE INFORMATION ABOUT GDGS

For more information about Generation Data Groups, consult the documentation at the following URLs:

```
https://www.ibm.com/docs/en/zos-basic-skills  
?topic=vtoc-what-is-generation-data-group
```

```
https://www.ibm.com/docs/en/zos/2.5.0  
?topic=guide-generation-data-sets
```

```
https://www.ibm.com/docs/en/zos/2.5.0  
?topic=files-processing-generation-data-groups
```

WHAT YOU KNOW AND WHAT YOU CAN DO

- You know a Generation Data Group (GDG) tracks multiple versions of the "same" data set.
- You know any one version of a data set in a GDG is called a Generation Data Set (GDS).
- You know a catalog entry for a GDG is created using the IDCAMS utility (Access Method Services)
- You can write JCL to create and catalog a GDG, to define a model DSCB for its GDSs, and to create and load GDSs.
- You can write JCL to retrieve any given GDS version and how to concatenate multiple GDSs (or all the GDSs in a GDG).
- You can write Ansible playbooks to work with GDGs on z/OS.
- You know where to look in the online IBM documentation for more information.

WHAT'S NEXT

- Working with VSAM